

DV2597 - Lab 2

Christoffer Bohman - chbh22@student.bth.se

March 20, 2026

1 Gauss-Jordan

1.1 Implementation

Gauss-Jordan is a continuation on the Gauss elimination algorithm which not only reduces a matrix to triangular form, but also further reduces it into a reduced row echelon form that can be used to present solutions to linear equations. The main difference between Gauss and Gauss-Jordan is the fact that the latter adds an additional elimination step that also handles the upper-right part of the matrix resulting in a diagonal with 1s and the rest of the matrix being 0s. How I implement the algorithm in CUDA is through a multitude of kernels that each complete a certain section of the algorithm in order for the whole algorithm to be thread-safe. More specifically how each kernel is constructed:

- Division
 - This kernel only handles the division of the affected row, similar to Gauss. It divides the entire row of elements.
- Post-Division
 - The Post-Division step as I call it, is the setting of the diagonal element to 1. This step is within a critical zone which entails that it has to be accessed with restrictions such as no more than one thread may write to the data at one time. It may seem odd to launch a kernel with only one block and one thread, however this is more than likely faster than copying the data to the cpu, doing the small calculation, and then copying it back to the gpu for further processing.
- Elimination 1
 - The first elimination is the last step of Gauss elimination where the matrix gets transformed into triangular form. The optimisations that have been employed include the usage of shared memory which results in faster memory accesses. This does however add slight complexity and the requirement of preparing the data by copying it into shared memory. This is done in parallel, as each thread has the responsibility for their own part of shared memory. The threads are then synced as to ensure the data has been prepared before using it.
- Elimination 2
 - The step that essentially replicates what Elimination 1 is doing, with the exception that the upper-right part of the matrix is being eliminated.

1.2 Measurements

Table 1: Measurements - Gauss-Jordan

Number of Threads	Number of Blocks	Average Time (s)
Sequential	N/A	0.611805
16 Threads	128 Blocks	0.554073
32 Threads	64 Blocks	0.41007
64 Threads	32 Blocks	0.358646
128 Threads	16 Blocks	0.291275

Table 1: Average execution times of the sequential and parallel variants of gauss-jordan respectively.

1.3 Analysis

The measurements show that CUDA in general performs between 10.5% and 110.0% faster depending on block/thread distribution. This is most likely due to the utilisation of parallelism and shared memory.

2 Odd-Even Sort

2.1 Implementation

The Odd-Even Sort is generally not the fastest sorting algorithm. However, due to the nature of the sorting, the algorithm is fit for massive parallelisation. The mechanism works by iteratively checking and swapping elements based on whether the index is even or odd, hence the name. This makes it possible to rather than sequentially, compute the sorting algorithm in parallel passes between odd and even. There are a couple of ways to implement this in CUDA, here is how I did it:

- Single Kernel
 - My implementation of the single block kernel uses two thread syncs as to ensure that the data doesn't corrupt or become invalid. Each thread gets an odd index that is based on its thread ID which then gets reassigned cyclically in order to satisfy the entire list and not just the first section. Then the algorithm starts by comparing the element at its index and the one to the left of it. This is done for all indices before syncing all threads. After that, a similar process is done for the right side. The process then repeats until it is fully sorted.
- Multiple Kernels
 - As for the implementation with multiple blocks, I chose to separate the sorting algorithm into two kernels. One that handles the comparison

with the left side, and one that handles the right side. This version abstracts away the need for thread syncing as the CPU handles the kernel launching which also implicates that the threads will be synced up before launching the next kernel. The cyclic assignment is also handled implicitly in the form of blocks.

2.2 Measurements

Table 2: Measurements - Odd-Even Sort

Number of Threads	Number of Blocks	Average Time (s)
Sequential	N/A	103.59671
Single Kernel (1024 Threads)	1 Block	39.704628
64 Threads	4096 Blocks	12.964334
128 Threads	2048 Blocks	8.134778
256 Threads	1024 Blocks	8.067528
512 Threads	512 Blocks	8.414528
1024 Threads	256 Blocks	12.341005

Table 1: Average execution times of the sequential and parallel variants of odd-even sort respectively.

2.3 Analysis

As seen in the measurements, we can see a 160.9% increase in performance for the single kernel variant and between 699.1% and 1184.1% performance increase for the multiple kernel variant. The single kernel variant suffers from major performance losses compared to the multi-kernel version most likely due to the restriction of threads that are active concurrently. The multiple kernels allow for more blocks to be working simultaneously.